RAG Architectures for AI Engineers

Interview Questions & Implementation Cheatsheets

Lamhot Siagian
PhD Student • AI Evaluation Engineer • AI Engineer
Machine Learning • Data Science & AI/ML
linkedin.com/in/lamhotsiagian
@growth.ideology • lamhotsiagian2025@gmail.com

Publication Month: February 2026

Disclaimer
This book is provided for educational purposes. It is not legal, medical, or financial advice. The examples and architectures are illustrative; adapt them to your data, security requirements, and production constraints.

Trademarks
All product names, trademarks, and registered trademarks are the property of their respective owners. Use of names does not imply endorsement.

Publication Month
February 2026

Contents

Chapter 1

Naive RAG

Interview Anchor

- **Sound bite:** "Naive RAG is Top-k dense retrieval + prompt stuffing + generation; correctness depends on retrieval quality and grounding."
- **Sound bite:** "If the chunk is missing, the model will guess. My job is to make retrieval reliable and the prompt force citations."
- **Sound bite:** "I tune chunking, embeddings, and k before I tune prompts."

Whiteboard framework (4 steps)

1. Ingest: clean, chunk, embed, and index documents.
2. Retrieve: embed query, run ANN search, return Top-k chunks.
3. Augment: build a grounded prompt with citations and constraints.
4. Generate: answer using only retrieved context; log traces for evaluation.

Example interview questions

- Conceptual: When does naive RAG fail even with a strong LLM, and why?
- Scenario: Your RAG answers are verbose but wrong for 20% of queries. What do you check first?

In interviews, naive RAG is the baseline used to test whether you can build a reliable retrieval loop. The interviewer is listening for concrete choices: chunk size, overlap, embedding model, and how you force grounding. A strong answer explains failure modes and how you would measure them.

Naive RAG retrieves documents purely by dense vector similarity between the query embedding and stored chunk embeddings. It is simple, fast, and often good enough for fact lookups when the query wording is close to the source language. Its weakness is that "semantic similarity" does not guarantee the chunk contains the needed fact.

In practice, most accuracy gains come from data prep rather than prompt tricks. I start with clean text, stable chunk boundaries, and consistent metadata like source, section title, and timestamps. Then I choose an embedding model aligned with the domain and language.

Finally, naive RAG needs guardrails so the LLM does not hallucinate when retrieval misses. I

use a prompt contract like "answer only from context" and require citations per claim. I also add an abstain path when confidence is low or sources conflict.

1.1 Core Pipeline

1.1.1 Ingestion and chunking

Chunking should preserve meaning boundaries. Use headings, paragraphs, and lists as primary split points, then enforce a token budget. Overlap helps recall for facts that span boundaries, but too much overlap increases index size and duplicates in Top-k.

1.1.2 Dense retrieval

Dense retrieval is typically ANN search over embeddings using FAISS, ScaNN, or a managed vector database. Start with cosine similarity or inner product, then measure recall@k on a labeled set. If recall@k is low, the generator cannot fix it.

1.1.3 Prompt assembly and grounding

A grounded prompt should include: user question, retrieved chunks with source IDs, and explicit constraints. I usually add: "If the answer is not in the context, say you don't know." I also ask for short citations like `[doc3#p2]` per sentence.

1.1.4 Observability and evaluation

Log the query, Top-k chunk IDs, similarity scores, and the final answer with citations. Evaluate retrieval (recall@k, MRR) and generation (faithfulness, answer correctness) separately. This separation makes debugging fast.

1.2 Implementation Notes

1.2.1 Minimal reference implementation (pseudocode)

```
# Offline: build index
for doc in corpus:
    chunks = chunk(doc.text, max_tokens=400, overlap=60)
    for c in chunks:
        vec = embed(c.text)
        index.add(vec, metadata={source_id, title, ts, chunk_id})

# Online: answer
q_vec = embed(user_query)
topk = index.search(q_vec, k=6) # returns (chunk_text, metadata, score)
prompt = build_prompt(user_query, topk, require_citations=True, allow_abstain=True)
answer = llm.generate(prompt)
return answer
```

1.2.2 Practical tuning checklist

Tune in this order: (1) chunking, (2) embedding model, (3) k and filters, (4) reranking, (5) prompting. Each step should show a measurable gain on a fixed eval set. If you cannot measure it, you are guessing.

1.3 Interview Questions and Answers

Q: What is naive RAG in one sentence?
A: Naive RAG embeds the query, retrieves Top-k semantically similar chunks by vector similarity, and prompts an LLM to answer using those chunks as context. It assumes the nearest chunks contain the needed facts. It is the simplest RAG baseline and a strong starting point for production.

Q: Why do naive RAG systems hallucinate?
A: Hallucination happens when retrieval misses or returns loosely related chunks, but the LLM still tries to be helpful. The fix is not only prompting; you need better retrieval recall and an explicit abstain policy. Logging Top-k evidence per answer makes this visible.

Q: How do you choose chunk size and overlap?
A: I align chunk size to the unit of meaning in the data and the model context budget, often 300–600 tokens for general text. I use overlap only to protect facts across boundaries, typically 10–20% of chunk length. Then I validate with recall@k and manual spot checks on hard queries.

Q: What metrics do you use to evaluate naive RAG?
A: I split metrics into retrieval and generation. Retrieval uses recall@k, MRR, and coverage of gold evidence. Generation uses correctness, faithfulness to citations, and an abstention rate that matches business risk.

Q: What is recall@k and why does it matter?
A: Recall@k measures whether the correct supporting chunk appears in the Top-k retrieved results. If recall@k is low, the LLM does not see the evidence and will guess. Improving recall@k is usually the highest ROI change early on.

Q: When is dense retrieval worse than keyword search?
A: Dense retrieval can fail for exact IDs, error codes, or rare names that embeddings smooth over. Keyword search (BM25) handles exact token matches and can be more stable for structured terms. In practice, I often add a hybrid or reranker if I see this pattern.

Q: How do you reduce duplicate chunks in Top-k?
A: Duplicates come from overlap and near-identical sections. I deduplicate by source ID and local similarity, or I use Maximal Marginal Relevance (MMR) to diversify results. This improves context efficiency without reducing recall.

Q: What is reranking and when do you add it?
A: Reranking uses a stronger model (often a cross-encoder) to reorder candidates returned by the fast retriever. I add it when recall is decent but precision is low, meaning Top-k contains the answer but not near the top. It is a targeted fix for relevance, not a substitute for ingestion quality.

Q: How do you enforce grounding in the generated answer?

A: I make citations mandatory, restrict the model to answer only from context, and add an abstain rule. I also separate "answer" and "evidence" fields so I can automatically check that every claim has supporting text. If the system cannot cite, it should not claim.

Q: What are the first three things you debug when answers are wrong?
A: First, check if the gold evidence is in Top-k; if not, it is a retrieval issue. Second, inspect chunk boundaries and metadata filters for accidental exclusions. Third, check whether the prompt allows free-form guessing or ignores citations.

1.4 Interview Cheatsheet

Interview Cheatsheet

Key terms (1-line)

- **Chunking:** splitting documents into retrieval units that preserve meaning.
- **Embedding:** vector representation used for similarity search.
- **ANN index:** approximate nearest neighbor structure for fast Top-k search.
- **Top-k:** number of chunks retrieved per query.
- **Recall@k:** whether the correct evidence appears in Top-k.
- **MRR:** ranking metric that rewards higher placement of the correct evidence.
- **MMR:** diversification method to reduce redundant retrieved chunks.
- **Reranker:** stronger relevance model applied to retrieved candidates.
- **Grounding:** forcing answers to be supported by provided evidence.
- **Abstention:** explicit "I don't know" path when evidence is missing.

Rapid-fire rehearsal

- **Q:** What is naive RAG? **A:** Top-k dense retrieval + grounded prompt + generation.
- **Q:** Why does it fail? **A:** Missing/irrelevant evidence; embeddings are not guarantees.
- **Q:** What do you tune first? **A:** Chunking and retrieval recall@k.
- **Q:** How do you prevent guessing? **A:** citations + abstain rule + eval on faithfulness.

"Tell me about a time..." story skeleton

- **Situation:** A knowledge-base chatbot gave confident but wrong answers.
- **Task:** Improve answer correctness without increasing latency too much.
- **Action:** Built a labeled eval set, fixed chunking, tuned k, added MMR and a citation-required prompt, then monitored recall@k and faithfulness.
- **Result:** Recall@6 increased, hallucinations dropped, and support tickets for wrong answers decreased measurably.

Chapter 2

Multimodal RAG

Interview Anchor

- **Sound bite:** "Multimodal RAG retrieves evidence across text, images, and audio; the hard part is a shared retrieval interface and aligned embeddings."
- **Sound bite:** "I store modality-aware metadata and route queries by intent, then fuse evidence into a single grounded context."
- **Sound bite:** "I treat OCR/ASR as ingestion, not generation."

Whiteboard framework (5 steps)

1. Ingest per modality: text chunking, image captions/OCR, audio ASR + timestamps.
2. Embed: modality-specific encoders or a shared joint encoder (e.g., CLIP-style).
3. Retrieve: per-modality Top-k plus optional reranking.
4. Fuse: align evidence by entity, time, or page/region coordinates.
5. Generate: use a VLM/LLM with citations back to each modality artifact.

Example interview questions

- Conceptual: What is the difference between joint-embedding retrieval and late fusion retrieval?
- Scenario: Users ask "What does this screenshot mean?" How do you retrieve and ground the answer?

In interviews, multimodal RAG tests whether you can reason about ingestion and retrieval beyond plain text. A good answer separates concerns: extraction (OCR/ASR), embedding, retrieval, and evidence fusion. The interviewer wants to hear how you keep provenance when evidence is a bounding box, a time segment, or a figure.

Multimodal RAG handles multiple data types by embedding and retrieving across modalities. The simplest design runs independent retrievers for each modality, then fuses the results into one context. More advanced designs use a joint embedding space so a text query can directly retrieve images.

In practice, modality alignment and metadata decide success. For images, I store caption text, OCR text, and region coordinates. For audio, I store transcript segments with timestamps so

answers can cite “00:31–00:44”.

Finally, multimodal generation must preserve grounding. I require citations that reference an artifact ID plus location (page/box/time). If the model cannot point to a region or time span, I treat the answer as ungrounded and fail it in evaluation.

2.1 Core Pipeline

2.1.1 Text retrieval

Text follows standard chunking and dense retrieval. The key is consistent document IDs shared with other modalities, such as the same PDF page IDs used for image regions.

2.1.2 Image retrieval

For image-heavy corpora, I use a vision encoder (CLIP-like) to embed images and optionally embed image captions. Retrieval can be text-to-image (query text embedding vs image embedding) or text-to-caption (query text vs caption embeddings). I always preserve a pointer to the original file and region.

2.1.3 Audio retrieval

Audio ingestion typically produces transcripts via ASR and optionally speaker diarization. Retrieval uses transcript chunks as text, but citations should include timestamp ranges. For meeting data, diarization labels help resolve who said what.

2.1.4 Evidence fusion

Fusion can be late (merge Top-k from each modality) or early (joint embeddings). Late fusion is easier to debug because you can see which modality produced each evidence piece. Early fusion can help recall for cross-modal queries but is harder to diagnose when it fails.

2.2 Implementation Notes

2.2.1 Routing and fusion (pseudocode)

```
intent = classify_query(user_query) # e.g., "needs_image", "needs_audio", "text_only"

evidence = []
if intent in ["text_only", "mixed"]:
    evidence += text_index.search(embed_text(user_query), k=4)

if intent in ["needs_image", "mixed"]:
    evidence += image_index.search(embed_text(user_query), k=3) # text->image

if intent in ["needs_audio", "mixed"]:
    evidence += audio_index.search(embed_text(user_query), k=3) # text->transcript

evidence = rerank(user_query, evidence) # optional cross-encoder over text surrogates
```

```
context = format_with_provenance(evidence) # includes page/box/time pointers
answer = vlm_or_llm.generate(build_prompt(user_query, context, require_citations=True))
```

2.2.2 Two common failure modes

First, OCR/ASR errors create wrong evidence, and retrieval cannot fix it. Second, provenance gets lost when you convert images or audio to text surrogates. I mitigate both by storing raw artifacts, extracted text, confidence scores, and location pointers.

2.3 Interview Questions and Answers

Q: What is multimodal RAG in one sentence?
A: Multimodal RAG retrieves and uses evidence from multiple modalities—text, images, audio, or video—to ground a generated answer. It adds ingestion steps like OCR and ASR and requires location-aware citations. The goal is cross-modal question answering with traceable evidence.

Q: How do you represent images for retrieval?
A: I embed images using a vision encoder and store the embedding plus metadata like file ID and bounding boxes. I also store captions and OCR text as searchable surrogates for hybrid retrieval. In production, I keep both because text surrogates are easier to rerank and explain.

Q: What is joint embedding vs late fusion?
A: Joint embedding maps text and images into the same vector space, enabling direct text-to-image retrieval. Late fusion retrieves separately per modality and merges results afterward. Joint embedding can improve cross-modal recall, while late fusion is simpler to debug and tune.

Q: How do you handle screenshots or UI images?
A: I run OCR to extract visible text and store bounding boxes for each text block. I also compute an image embedding so queries like "error dialog" can match visual patterns. Answers cite both the OCR snippet and the bounding box location.

Q: How do you ground answers to audio?
A: I treat ASR transcripts as the retrieval unit, but I require citations that include timestamp ranges. If a claim cannot be tied to a segment like "02:10–02:24", it is not grounded. For long audio, I chunk by pauses and speaker turns to improve precision.

Q: What does modality routing look like in practice?
A: I classify the query intent using rules or a lightweight model: text-only, needs-image, needs-audio, or mixed. Routing controls which indices are queried and how much context budget is allocated per modality. This keeps latency stable and reduces irrelevant evidence.

Q: What is the biggest engineering risk in multimodal RAG?
A: Provenance and alignment are the biggest risks. If you lose page, region, or time pointers, you cannot validate or cite evidence. I design data models so every extracted text span has a stable link back to the raw artifact and location.

Q: How do you evaluate multimodal RAG?
A: I evaluate extraction quality (OCR/ASR error rate), retrieval quality per modality (recall@k), and generation faithfulness with location-aware citations. I also run modality ablation tests to see if the system truly uses images or audio. This avoids "fake multimodality" where text dominates.

Q: When would you avoid multimodal RAG?
A: If your user questions are entirely text-based and your documents have reliable text versions, multimodal adds cost without benefit. OCR and image embeddings increase ingestion time and storage. I only add multimodality when users routinely need figures, screenshots, diagrams, or spoken content.

Q: How do you prevent the model from inventing visual details?
A: I require the model to quote OCR text or refer to a cited region for each visual claim. I also add a constraint like "do not describe unseen elements" and include the extracted fields explicitly. If evidence is missing, the model must abstain or request the image.

2.4 Interview Cheatsheet

Interview Cheatsheet

Key terms (1-line)

- **VLM:** vision-language model used for multimodal reasoning and generation.
- **OCR:** text extraction from images with bounding boxes and confidence scores.
- **ASR:** speech-to-text transcription, often chunked by timestamps.
- **Joint embedding:** shared vector space for text and images.
- **Late fusion:** merge evidence after per-modality retrieval.
- **Provenance:** artifact + location pointer (page/box/time) for citations.
- **Modality routing:** deciding which indices to query based on intent.
- **Ablation test:** remove a modality to measure its real contribution.

Rapid-fire rehearsal

- **Q:** How do you ground image answers? **A:** OCR + region IDs + citations.
- **Q:** Audio grounding? **A:** transcript chunks with timestamp citations.
- **Q:** Joint vs late fusion? **A:** shared space vs merge-after-retrieval.
- **Q:** Biggest risk? **A:** losing provenance and alignment.

"Tell me about a time..." story skeleton

- **Situation:** Support engineers needed answers from screenshots and meeting recordings.
- **Task:** Add multimodal grounding without breaking latency and traceability.
- **Action:** Added OCR + image embeddings, ASR with timestamps, implemented query routing, and enforced location-based citations in prompts and evaluation.
- **Result:** Improved resolution time for visual/audio issues and enabled reviewers to click evidence locations for verification.

Chapter 3

HyDE (Hypothetical Document Embeddings)

Interview Anchor

- **Sound bite:** "HyDE fixes embedding mismatch by generating a hypothetical answer document, embedding it, then retrieving real evidence."
- **Sound bite:** "It is a retrieval trick, not a generation trick; I still ground the final answer on retrieved sources."
- **Sound bite:** "HyDE helps when user queries are short, vague, or use different vocabulary than the corpus."

Whiteboard framework (4 steps)

1. Draft: LLM writes a hypothetical "ideal" document for the query.
2. Embed: compute embedding of the hypothetical text (and optionally the original query).
3. Retrieve: search the vector index using HyDE embedding; collect candidates.
4. Verify: rerank and generate using only retrieved evidence with citations.

Example interview questions

- Conceptual: Why can HyDE improve recall when query embeddings fail?
- Scenario: You have short queries like "refund policy" and long policy docs. How would you use HyDE safely?

In interviews, HyDE is a signal that you understand embedding failure cases. The interviewer expects you to explain why query embeddings may be far from relevant documents. A good answer also explains how you control risk so the hypothetical text does not bias the final answer.

HyDE is used when the user query is not semantically similar to how documents are written. Instead of embedding the raw query, the system generates a hypothetical answer document that uses the vocabulary and structure likely present in the corpus. The embedding of this generated text becomes a better retrieval key.

In practice, HyDE is most useful for short, underspecified queries and domain-specific corpora

with formal language. It can also help for cross-lingual or jargon-heavy domains when users use everyday words. The key is that HyDE improves retrieval recall, not the truthfulness of generation.

HyDE must be paired with strict grounding. I never let the hypothetical document appear as evidence to the user. I use it only to retrieve real sources, then generate from retrieved text with citations and an abstain policy.

3.1 Core Pipeline

3.1.1 Generating the hypothetical document

The hypothetical document should be short, factual in tone, and structured like the corpus. I usually prompt for a "one-page internal note" or "policy excerpt" style. The goal is to produce text that shares terms with relevant chunks.

3.1.2 Retrieval and blending

A common pattern is to retrieve with the HyDE embedding and also retrieve with the raw query embedding. Then you merge candidates and rerank. This reduces the risk that a bad hypothetical draft pulls you away from correct evidence.

3.1.3 Reranking and grounding

Reranking is important because HyDE can broaden the candidate set. I rerank using a cross-encoder over (query, chunk) pairs, then keep the highest-precision evidence. The final LLM prompt must cite only retrieved chunks.

3.2 Implementation Notes

3.2.1 HyDE retrieval (pseudocode)

```
hyde_text = llm.generate(
  "Write a concise, factual document that would answer: " + user_query
)

v_hyde = embed(hyde_text)
v_query = embed(user_query)

cand = index.search(v_hyde, k=20) + index.search(v_query, k=20)
cand = dedupe(cand)
top = rerank_cross_encoder(user_query, cand)[:6]

prompt = build_prompt(user_query, top, require_citations=True, allow_abstain=True)
answer = llm.generate(prompt)
```

3.2.2 Safety and cost controls

HyDE adds an extra LLM call and may increase latency. I cache HyDE drafts for repeated queries and cap the draft length. I also keep logs so I can spot when HyDE causes topic drift.

3.3 Interview Questions and Answers

Q: What problem does HyDE solve?
A: HyDE addresses embedding mismatch when a short query does not land near the right documents in vector space. It generates a hypothetical document that uses corpus-like language, then embeds that document for retrieval. The intent is higher recall, especially for vague queries.

Q: How is HyDE different from query expansion?
A: Query expansion adds keywords or paraphrases to the query, often using rules or an LLM. HyDE generates a longer pseudo-document that resembles the target corpus, which can shift the embedding more strongly. I still validate via reranking and grounding because both can introduce drift.

Q: When does HyDE work best?
A: It works best when queries are short and documents are long and formal, such as policies, manuals, or research notes. It also helps when users use different synonyms than the corpus. If the corpus is noisy or highly diverse, HyDE gains are smaller.

Q: What are the risks of HyDE?
A: The hypothetical document can hallucinate details and pull retrieval toward the wrong topic. It also adds cost and latency due to an extra generation step. I mitigate this by blending with raw query retrieval and using reranking to enforce relevance.

Q: Should the hypothetical document be shown to users?
A: No, I treat it as an internal retrieval artifact. Showing it risks presenting fabricated content as evidence. Users should only see citations to real retrieved sources.

Q: How do you prompt the HyDE draft?
A: I keep it concise and corpus-shaped: "Write a factual internal note that answers X, using terms found in technical documentation." I avoid asking for specific numbers unless the corpus likely contains them. I cap length so it stays focused and cheap to embed.

Q: How do you evaluate HyDE?
A: I compare retrieval metrics like recall@k and MRR with and without HyDE on a fixed dataset. I also check downstream correctness and faithfulness, because better recall can still produce noisy context. The key question is whether HyDE improves evidence coverage on the hard queries.

Q: How do you combine HyDE and standard retrieval?
A: I retrieve candidates using both HyDE and raw query embeddings, merge, dedupe, and rerank. This makes HyDE a recall booster without making it the only path. In practice, it stabilizes performance across query types.

Q: Does HyDE replace reranking?
A: No, HyDE often increases candidate diversity, so reranking becomes more important. Reranking improves precision by scoring candidates against the original query. HyDE and reranking are complementary: recall first, precision second.

Q: How do you control topic drift?
A: I monitor the semantic distance between query and HyDE draft and flag outliers. I also enforce that final answers cite retrieved evidence and can abstain. If drift is common, I tighten the HyDE prompt or reduce its weight in candidate fusion.

3.4 Interview Cheatsheet

Interview Cheatsheet

Key terms (1-line)

- **Embedding mismatch:** query and relevant docs are far apart in vector space.
- **HyDE draft:** hypothetical corpus-like text generated from the query.
- **Candidate fusion:** merging results from multiple retrievers before reranking.
- **Topic drift:** retrieval shifts to the wrong subject due to a bad draft.
- **Cross-encoder reranker:** relevance model over (query, chunk) pairs.
- **Grounding contract:** answer only from retrieved evidence with citations.

Rapid-fire rehearsal

- **Q:** HyDE in one line? **A:** Generate pseudo-doc, embed it, retrieve real docs.
- **Q:** When use it? **A:** short/vague queries, formal corpora, synonym mismatch.
- **Q:** Main risk? **A:** topic drift from hallucinated draft.
- **Q:** Mitigation? **A:** blend with raw query + rerank + strict citations.

"Tell me about a time..." story skeleton

- **Situation:** Users typed two-word queries, and retrieval missed key policies.
- **Task:** Improve evidence recall without changing the document store.
- **Action:** Implemented HyDE drafts, fused candidates with standard retrieval, added reranking, and measured recall@k improvements on a labeled set.
- **Result:** Higher evidence coverage on short queries and fewer "confident wrong" answers due to better grounding and abstention.

Chapter 4

Corrective RAG

Interview Anchor

- **Sound bite:** "Corrective RAG adds a verification loop: retrieve, check against trusted sources, then correct or abstain."
- **Sound bite:** "I separate evidence collection from answer writing, and I fail the answer if citations cannot be validated."
- **Sound bite:** "Freshness is a data problem; I add time-aware retrieval and source priorities."

Whiteboard framework (5 steps)

1. Retrieve: get candidates from internal store.
2. Validate: detect staleness, conflicts, or low relevance.
3. Augment: pull trusted sources (curated KB, web search, or authoritative APIs).
4. Reconcile: choose consistent evidence set; flag conflicts.
5. Generate: answer with verified citations or abstain.

Example interview questions

- Conceptual: What does "verification" mean in Corrective RAG, and what are the failure modes?
- Scenario: Your internal docs say policy X, but a newer public FAQ says policy Y. What do you do?

In interviews, Corrective RAG tests production judgment: accuracy, freshness, and risk controls. The interviewer wants to hear how you detect bad retrieval, how you pick trusted sources, and how you handle conflicts. A strong answer explains what the system does when sources disagree.

Corrective RAG validates retrieved results before letting the LLM answer. It is designed for domains where internal corpora can be stale or incomplete, such as product docs, compliance policies, or dynamic pricing rules. The key idea is that retrieval is not automatically trustworthy.

In practice, I add explicit checks on retrieved evidence. I look at similarity scores, metadata timestamps, and contradiction signals across chunks. If evidence is weak or conflicting, I expand retrieval to trusted sources like a curated database, official web pages, or a structured API.

Corrective RAG must have a clear conflict policy. Sometimes the right behavior is to prefer authoritative sources; other times it is to surface the conflict and ask for clarification. In all cases, the model should not merge contradictions into a single confident answer.

4.1 Core Pipeline

4.1.1 Validation signals

Validation can use: minimum similarity thresholds, entity overlap checks, and content-based contradiction detection. For time-sensitive domains, timestamps and version IDs are high-signal. I also use a "coverage" heuristic: did retrieval return at least one chunk that directly answers the question?

4.1.2 Trusted source augmentation

Trusted sources can be internal (gold docs, approved FAQs) or external (official websites, standards bodies, regulatory APIs). The retrieval method depends on the source: vector search for text, keyword search for exact phrases, and API calls for structured facts. The key is to track provenance and priority.

4.1.3 Reconciliation strategies

Common strategies include source ranking (prefer official), majority vote across sources, or asking the user when ambiguity remains. For regulated domains, I bias toward abstention when evidence is inconsistent. I also log conflicts so content owners can fix the corpus.

4.2 Implementation Notes

4.2.1 Verification loop (pseudocode)

```
cand = internal_index.search(embed(user_query), k=8)
signals = validate(cand) # relevance, freshness, conflicts

if signals["low_coverage"] or signals["stale"] or signals["conflict"]:
    trusted = trusted_search(user_query) # curated KB or web/API
    merged = merge_evidence(cand, trusted)
else:
    merged = cand

final = rerank(user_query, merged)[:6]
prompt = build_prompt(user_query, final, require_citations=True, allow_abstain=True,
                      conflict_policy="surface_or_abstain")
answer = llm.generate(prompt)
return answer
```

4.2.2 Practical guardrails

I treat verification as a first-class stage with its own metrics. I track "citation validity rate" and "conflict rate" by topic. If verification frequently triggers web/API calls, I cache results with TTL to manage latency.

4.3 Interview Questions and Answers

Q: What is Corrective RAG in one sentence?
A: Corrective RAG retrieves evidence, validates it against trusted sources or consistency checks, and then corrects, filters, or abstains before answering. It is designed to reduce errors from stale or low-quality retrieval. The output is an answer backed by verified citations.

Q: How do you define a "trusted source"?
A: A trusted source is one with clear ownership, versioning, and authority for the domain, such as an official policy page or a curated internal KB. Trust is not absolute; it is prioritized by topic and risk. I encode this as source ranks and allowed domains.

Q: What are common validation checks?
A: I check relevance (scores and reranker), freshness (timestamps/version IDs), and conflicts (contradictions across chunks). I also check coverage: does any chunk contain the key entities and answer pattern. These checks decide whether to expand retrieval or abstain.

Q: How do you handle conflicting evidence?
A: First, I try to find the most authoritative and most recent source. If I cannot resolve it, I either surface the conflict with citations or abstain and ask for a clarifying constraint. The worst option is to average conflicting statements into one confident answer.

Q: When should Corrective RAG call web search or APIs?
A: When the question is time-sensitive, when internal evidence is stale, or when validation signals are weak. I also use external sources when the user explicitly asks for "latest" information. I control cost with caching and source whitelists.

Q: How do you keep verification from becoming a latency bottleneck?
A: I make verification conditional based on signals and query intent. I cache external lookups with TTL and batch calls when possible. I also use lightweight checks first, reserving expensive calls for high-risk queries.

Q: What does "citation validity" mean?
A: It means every cited snippet exists, is correctly referenced, and supports the claimed statement. I check this by storing chunk hashes, IDs, and offsets, then validating that cited spans appear in the retrieved context. Invalid citations are treated as a failure.

Q: How do you evaluate Corrective RAG?
A: I measure accuracy and faithfulness on a labeled set, plus operational metrics like verification trigger rate and external call latency. I also track conflict rate and abstention rate, because a safe system should abstain more when evidence is weak. The goal is fewer high-confidence errors.

Q: What is a good prompt contract for Corrective RAG?

A: I instruct the model to answer only from verified context and to cite sources per claim. I include a conflict rule: "If sources disagree, say so and cite both." I also include an abstain rule for missing evidence.

Q: How do you prevent verification from drifting to low-quality sources?
A: I use allowlists and a ranked source registry rather than open web crawling. I also enforce domain constraints and prefer structured APIs when available. Evidence from unranked sources is treated as supplemental, not authoritative.

4.4 Interview Cheatsheet

Interview Cheatsheet

Key terms (1-line)

- **Verification loop:** extra stage to validate and correct retrieved evidence.
- **Freshness:** evidence recency via timestamps, versions, or TTL.
- **Conflict policy:** rules for handling contradictory sources.
- **Citation validity:** citations must exist and support the claim.
- **Source registry:** prioritized list of allowed sources by topic.
- **TTL cache:** time-limited caching of external lookups.

Rapid-fire rehearsal

- **Q:** Why Corrective RAG? **A:** internal docs can be stale; verify before answering.
- **Q:** Trigger signals? **A:** low coverage, staleness, conflicts, "latest" intent.
- **Q:** Conflict behavior? **A:** prefer authoritative+recent, else surface or abstain.
- **Q:** Key metric? **A:** citation validity and high-confidence error rate.

"Tell me about a time..." story skeleton

- **Situation:** A support bot used outdated internal docs and gave wrong policy guidance.
- **Task:** Reduce high-confidence mistakes for time-sensitive questions.
- **Action:** Added validation signals, a trusted-source registry, conditional web/API augmentation, and a conflict policy with mandatory citations and abstention.
- **Result:** High-confidence error rate dropped, and policy changes were reflected faster without manual reindexing.

Chapter 5

Graph RAG

Interview Anchor

- **Sound bite:** "Graph RAG turns text into entities and relationships so retrieval can follow links, not just similarity."
- **Sound bite:** "I use the graph to find the right neighborhood, then pull original text as evidence."
- **Sound bite:** "A graph improves multi-hop questions because it makes relationships explicit."

Whiteboard framework (5 steps)

1. Extract: NER + relation extraction to produce triples (subject, predicate, object).
2. Build: store triples in a graph DB with stable IDs and provenance links to text spans.
3. Query: map user query to entities and constraints; retrieve a relevant subgraph.
4. Expand: traverse edges (k-hop) to collect connected entities and candidate facts.
5. Ground: fetch supporting text chunks for the selected triples; answer with citations.

Example interview questions

- Conceptual: Why does a knowledge graph help multi-hop retrieval compared to vectors alone?
- Scenario: "Which teams depend on service X and who owns the on-call rotation?" How would Graph RAG answer?

In interviews, Graph RAG checks whether you can structure messy text into something queryable. The interviewer expects you to talk about entity resolution, provenance, and graph queries. A strong answer also admits the trade-offs: extraction errors, maintenance, and schema drift.

Graph RAG converts retrieved or ingested content into a knowledge graph to capture relationships and entities. Instead of retrieving only by similarity, the system can traverse edges like *service* $\rightarrow$ *team* $\rightarrow$ *owner*. This supports multi-step reasoning and reduces reliance on the LLM to infer relationships from raw paragraphs.

In practice, I treat the graph as an index over relationships, not a replacement for text evidence. The graph finds candidates, but the final answer cites the original sentences that support each fact. This approach keeps the system auditable and reduces errors from extraction mistakes.

Graph RAG works best when your domain has stable entities and repeated relations, such as infrastructure inventories, org charts, product catalogs, or biomedical knowledge. If documents are short-lived and entity names are unstable, you may spend more time on resolution than you gain in retrieval.

5.1 Core Pipeline

5.1.1 Entity and relation extraction

Extraction can be rule-based, ML-based, or LLM-assisted. I start with a narrow schema of high-value relations, then expand. I also store confidence scores and the source text span for each triple.

5.1.2 Graph storage and provenance

A graph database like Neo4j, Neptune, or a property-graph layer on top of a key-value store can work. The critical piece is provenance: every node and edge should link back to source document IDs and offsets. Without provenance, you cannot validate or cite facts.

5.1.3 Querying and traversal

A user query is mapped to seed entities and constraints. Then the system retrieves a subgraph via k-hop expansion, filtered by relation types. The subgraph becomes a structured context, and the system fetches the supporting text for the selected edges.

5.2 Implementation Notes

5.2.1 Minimal extraction + query flow (pseudocode)

```
# Ingest
for chunk in chunks:
    ents = ner(chunk.text)
    rels = relation_extract(chunk.text, ents) # triples with confidence
    for (s,p,o,conf,span) in rels:
        graph.upsert_edge(s, p, o, props={conf, doc_id, span})

# Query
seeds = entity_link(user_query, graph)
subgraph = graph.k_hop(seeds, k=2, rel_types=allowed_types)
facts = select_high_confidence(subgraph)
evidence = fetch_supporting_text(facts) # doc_id + span -> snippet

answer = llm.generate(build_prompt(user_query, evidence, require_citations=True))
```

5.2.2 Entity resolution in practice

Entity resolution is often the hardest part. I use canonical IDs, alias tables, and normalization (case, punctuation, abbreviations). I also log unresolved entities so the schema and alias lists improve over time.

5.3 Interview Questions and Answers

Q: What is Graph RAG in one sentence?
A: Graph RAG builds or uses a knowledge graph of entities and relationships so retrieval can traverse links and support multi-hop questions. It uses the graph to locate relevant facts, then fetches supporting text for grounding. The output is a structured, auditable evidence set.

Q: Why not just let the LLM reason over text?
A: LLMs can infer relationships, but they are unreliable when evidence is distributed across many documents. A graph makes relationships explicit and queryable, so you can retrieve the right connected facts quickly. It also improves auditability because each edge can be traced to a source span.

Q: What is the biggest risk in Graph RAG?
A: Extraction errors and entity resolution errors can create incorrect edges. If you treat the graph as truth, you will amplify mistakes. I mitigate by storing confidence, requiring provenance, and using text evidence in the final answer.

Q: How do you keep provenance for graph facts?
A: Every node and edge stores a reference to `doc_id`, `chunk_id`, and a character or token span. When answering, I fetch the original text snippet for each claimed fact. This makes citations precise and allows automated validation.

Q: What graph schema do you start with?
A: I start with a small set of high-value entity types and relation types aligned to the product, like `Service-owned_by->Team`. Then I add only relations that show repeated query demand. A narrow schema reduces extraction ambiguity and maintenance cost.

Q: How do you answer multi-hop questions with Graph RAG?
A: I map the query to seed entities, traverse a limited number of hops, and filter by relation types. Then I select candidate paths that satisfy constraints like ownership and dependency. Finally, I pull the text evidence for each edge on the chosen path and generate with citations.

Q: When would you avoid Graph RAG?
A: If entities are not stable or relations are rarely reused, the graph will be expensive to build and maintain. For ad-hoc, narrative documents, dense retrieval plus reranking may be enough. I also avoid it when provenance cannot be reliably captured.

Q: How do you evaluate a Graph RAG system?
A: I evaluate extraction quality (precision/recall of triples), entity linking accuracy, and question-answer correctness. I also test path retrieval on multi-hop benchmarks and track how often answers cite valid source spans. Graph quality metrics matter because they directly affect retrieval.

Q: How do you handle updates or deletions in source docs?
A: I treat the graph as derived data and rebuild or incrementally update it from a document version store. Edges carry `doc_version` and timestamps, so stale edges can be pruned. I also run periodic consistency checks to remove orphaned references.

Q: How do you integrate graph context into the prompt?
A: I provide a small set of candidate facts as triples plus the supporting text snippets. The triples give structure; the snippets provide grounding. This reduces token usage compared to dumping entire documents and makes citations straightforward.

5.4 Interview Cheatsheet

Interview Cheatsheet

Key terms (1-line)

- **Knowledge graph:** nodes (entities) and edges (relations) representing facts.
- **Triple:** (subject, predicate, object) representation of a relation.
- **Entity linking:** mapping a mention in text to a canonical graph ID.
- **k-hop traversal:** expand from seed nodes by following edges k steps.
- **Provenance span:** pointer from a node/edge back to source text offsets.
- **Schema:** allowed entity types and relation types.

Rapid-fire rehearsal

- **Q:** Why Graph RAG? **A:** makes relations explicit for multi-hop retrieval.
- **Q:** Biggest risk? **A:** bad edges from extraction/entity resolution.
- **Q:** Mitigation? **A:** confidence + provenance + cite original text.
- **Q:** Graph role? **A:** locate candidates; text remains evidence of record.

"Tell me about a time..." story skeleton

- **Situation:** Engineers asked dependency and ownership questions across many docs.
- **Task:** Reduce time to find correct owners and upstream/downstream impacts.
- **Action:** Built a small graph schema, implemented entity linking, stored provenance spans, and used k-hop traversal to assemble evidence for answers with citations.
- **Result:** Faster, more reliable multi-hop answers and clearer audit trails for operational decisions.

Chapter 6

Hybrid RAG

Interview Anchor

- **Sound bite:** "Hybrid RAG combines complementary retrievers—dense, sparse (BM25), and/or graph—then fuses and reranks."
- **Sound bite:** "I use hybrid when queries mix exact tokens and semantics, like IDs plus natural language."
- **Sound bite:** "Fusion is a ranking problem: retrieve broadly, then select precise evidence."

Whiteboard framework (5 steps)

1. Retrieve candidates from multiple retrievers (dense, BM25, graph).
2. Normalize scores or ranks across retrievers.
3. Fuse lists (e.g., RRF) and deduplicate by source and span.
4. Rerank with a stronger model for final Top-k evidence.
5. Generate grounded answer with citations and abstention.

Example interview questions

- Conceptual: Why does combining BM25 and dense retrieval often outperform either alone?
- Scenario: Users search for "error 0x80070005 permission denied" in a large wiki. How do you design retrieval?

In interviews, Hybrid RAG is where you show practical retrieval engineering. The interviewer wants you to justify why one retriever is insufficient and how you combine them without making the system brittle. A good answer explains fusion, reranking, and evaluation methodology.

Hybrid RAG combines dense vector retrieval with other retrieval types in a single pipeline. The most common hybrid is dense + BM25 because they fail in different ways: dense helps semantic paraphrases, BM25 helps exact tokens. Some systems also add graph retrieval for relationship-heavy domains.

In practice, hybrid is about coverage first, precision second. I retrieve more candidates than I will use, then fuse and rerank to pick the best evidence. This approach reduces the chance that a single retriever misses the critical chunk.

Hybrid systems must be measurable. I evaluate each retriever independently, then measure the incremental lift from fusion and reranking. If fusion adds complexity without accuracy gains, I simplify the pipeline.

6.1 Core Pipeline

6.1.1 Dense + sparse retrieval

Dense retrieval captures semantic similarity; BM25 captures lexical overlap and exact terms. I run both and merge candidates. For structured terms like IDs, I often boost BM25 results or apply filters.

6.1.2 Graph retrieval as a third channel

Graph retrieval can provide relationship constraints and candidate entities. I use it when queries require multi-hop reasoning or ownership/dependency chains. Graph results are converted back to text evidence for grounding.

6.1.3 Fusion and reranking

A simple, robust fusion method is Reciprocal Rank Fusion (RRF), which combines ranks rather than raw scores. After fusion, I apply a cross-encoder reranker to improve precision. Reranking is where many hybrid gains become real.

6.2 Implementation Notes

6.2.1 Reciprocal Rank Fusion (RRF)

RRF score for a document is typically $\sum_{r \in \text{retrievers}} \frac{1}{k_0 + \text{rank}_r}$, where k_0 is a constant like 60. It is stable because it does not require score calibration across retrievers. In interviews, calling out RRF is a strong signal because it is simple and effective.

6.2.2 Hybrid retrieval (pseudocode)

```
dense = dense_index.search(embed(query), k=30) # returns ranked list
sparse = bm25.search(query, k=30)
graph = graph_search(query, k=30) # optional

fused = rrf_fuse([dense, sparse, graph], k0=60)
fused = dedupe_by_source_span(fused)
top = rerank_cross_encoder(query, fused)[:6]

answer = llm.generate(build_prompt(query, top, require_citations=True, allow_abstain=
    True))
```

6.3 Interview Questions and Answers

Q: What is Hybrid RAG in one sentence?
A: Hybrid RAG combines multiple retrieval methods—often dense vectors, BM25 keyword search, and sometimes graph traversal—then fuses and reranks results to select high-quality evidence. It improves both semantic coverage and exact-match precision. The final answer is grounded on the fused evidence set.

Q: Why do BM25 and dense retrieval complement each other?
A: BM25 excels at exact tokens like error codes, names, and identifiers, while dense retrieval handles paraphrases and synonymy. Dense can miss rare terms; BM25 can miss semantically similar phrasing. Combining them reduces the chance of retrieval blind spots.

Q: How do you fuse results from different retrievers?
A: I often use rank-based fusion like RRF because it avoids score calibration issues. Then I deduplicate and rerank for precision. This gives broad coverage without letting noisy candidates dominate the final context.

Q: When is hybrid not worth it?
A: If your corpus is clean and queries are consistent, dense retrieval with reranking might already be sufficient. Hybrid adds operational complexity: more infrastructure and more tuning. I only keep it if offline evaluation shows a clear accuracy lift on representative queries.

Q: What is score calibration and why is it hard?
A: Dense retrievers and BM25 produce scores on different scales and distributions. Directly mixing raw scores can be misleading and unstable across corpora. Rank-based fusion or normalization is safer unless you invest in calibration.

Q: How do you handle duplicates across retrievers?
A: I deduplicate by a stable key like `doc_id + span` or `chunk_hash`. I also prefer diversity with MMR after fusion if the context is redundant. This improves context efficiency and reduces repeated evidence.

Q: How do you evaluate hybrid retrieval?
A: I measure recall@k and MRR per retriever, then measure the fused system against the same labeled set. I also track precision of the final Top-k after reranking. The key is incremental lift: hybrid should improve hard cases, not just average cases.

Q: How does graph retrieval fit into a hybrid pipeline?
A: Graph retrieval provides entity and relationship constraints, like owners, dependencies, or hierarchies. I use it to generate candidate nodes or paths, then fetch supporting text snippets for grounding. In the final context, the model sees text evidence, not just triples.

Q: What is a good default for candidate sizes?
A: I retrieve a larger candidate pool per retriever (e.g., 20–50) and then rerank down to a small final Top-k (e.g., 4–8). Candidate size depends on latency and corpus size, but the principle is consistent: recall first, then precision. I tune these using offline evaluation and latency budgets.

Q: How do you explain hybrid benefits in an interview?
A: I say: "Dense covers semantics, BM25 covers exact tokens, graph covers relationships; fusion gives coverage and reranking restores precision." Then I give a concrete example like error codes or

product part numbers. Finally, I mention metrics: recall@k lift and reduced high-confidence wrong answers.

6.4 Interview Cheatsheet

Interview Cheatsheet

Key terms (1-line)

- **BM25:** sparse keyword retrieval using term frequency and inverse document frequency.
- **Dense retrieval:** ANN search over embeddings for semantic similarity.
- **Fusion:** combining ranked lists from multiple retrievers.
- **RRF:** rank-based fusion method that avoids score calibration.
- **Candidate pool:** larger set retrieved before reranking.
- **Calibration:** mapping different retriever scores to a comparable scale.

Rapid-fire rehearsal

- **Q:** Why hybrid? **A:** different retrievers fail differently; combine for coverage.
- **Q:** Default fusion? **A:** RRF + dedupe + rerank.
- **Q:** When BM25 wins? **A:** IDs, error codes, rare names.
- **Q:** How prove value? **A:** offline recall@k/MRR lift on hard queries.

"Tell me about a time..." story skeleton

- **Situation:** A RAG assistant struggled with error-code and policy questions.
- **Task:** Improve retrieval precision without reducing semantic recall.
- **Action:** Added BM25 alongside dense, fused with RRF, reranked with a cross-encoder, and measured gains on a labeled query set.
- **Result:** Higher evidence recall for exact terms and fewer irrelevant chunks in the final context, improving answer correctness.

Chapter 7

Adaptive RAG

Interview Anchor

- **Sound bite:** "Adaptive RAG routes queries: simple questions use one-shot retrieval; complex questions trigger decomposition and iterative retrieval."
- **Sound bite:** "I optimize for cost and correctness by deciding the minimum retrieval work needed per query."
- **Sound bite:** "The router is a model: I evaluate it like any other component."

Whiteboard framework (5 steps)

1. Classify intent and complexity (fact lookup vs multi-hop vs procedural).
2. Choose strategy (single retrieval, multi-retriever, or decomposition).
3. Execute retrieval plan (sub-queries, filters, reranking).
4. Synthesize with grounding and conflict checks.
5. Stop or iterate based on confidence signals.

Example interview questions

- Conceptual: What signals tell you a query needs decomposition?
- Scenario: "Compare policy A vs policy B and recommend which to use for a small team." How do you design the plan?

In interviews, Adaptive RAG tests whether you can build systems that scale across query types. The interviewer wants to see routing logic, stopping criteria, and cost controls. A good answer also describes how you prevent infinite loops and how you measure router mistakes.

Adaptive RAG dynamically decides whether a query needs simple retrieval or a multi-step reasoning chain. Many real workloads are mixed: some queries are direct lookups, others require comparisons, summaries, or multi-hop dependency reasoning. Adaptive routing avoids running expensive multi-step pipelines on every query.

In practice, I implement a router that predicts the retrieval strategy. The router can be rule-based (keywords like "compare" or "how do I") or model-based (small classifier or LLM call). I keep the router simple first and evaluate it on labeled query types.

Adaptive RAG also needs clear stop conditions. I use confidence signals like evidence coverage, reranker scores, and contradiction detection. If evidence is still weak after one iteration, the system should either ask a clarifying question or abstain.

7.1 Core Pipeline

7.1.1 Query routing

Routing decides which pipeline to run. Common routes include: one-shot dense retrieval, hybrid retrieval, or decomposition into sub-queries. I also route based on risk, such as requiring corrective verification for "latest" queries.

7.1.2 Decomposition

Decomposition breaks a complex question into sub-questions that can be answered from evidence. For example, "compare A vs B" decomposes into "what is A", "what is B", and "key differences". Each sub-query retrieves evidence, then the system composes the final answer.

7.1.3 Stopping criteria

Stopping criteria prevent runaway tool use. I cap iterations and require an improvement signal to continue, such as higher evidence coverage or reduced uncertainty. Without a stopping rule, adaptive systems become unpredictable and costly.

7.2 Implementation Notes

7.2.1 Router + iterative retrieval (pseudocode)

```
route = router.predict(query) # "oneshot", "hybrid", "decompose"

if route == "oneshot":
    evidence = dense_index.search(embed(query), k=6)

elif route == "hybrid":
    evidence = fused_hybrid_retrieval(query)

else: # decompose
    subqs = decompose(query) # ["Q1", "Q2", ...]
    evidence = []
    for sq in subqs:
        evidence += dense_index.search(embed(sq), k=4)
    evidence = dedupe(evidence)

evidence = rerank(query, evidence)[:8]
signals = coverage_and_conflict_checks(query, evidence)

if signals["low_coverage"]:
    return ask_clarifying_or_abstain(query, evidence)
```

```
return llm.generate(build_prompt(query, evidence, require_citations=True))
```

7.2.2 How I label data for routing

I sample real queries and label them by strategy that produces correct answers at acceptable latency. Then I train or tune the router to predict that label. This keeps routing grounded in production constraints rather than intuition.

7.3 Interview Questions and Answers

Q: What is Adaptive RAG in one sentence?
A: Adaptive RAG chooses the retrieval strategy per query, ranging from one-shot retrieval to decomposed multi-step retrieval, based on complexity and risk. It aims to improve accuracy while controlling latency and cost. The system uses confidence signals to decide when to stop, iterate, or abstain.

Q: What signals indicate a query needs decomposition?
A: Signals include comparison words (compare, vs), multi-constraint requests (filter + rank), and questions that imply multiple entities or steps. Another signal is low evidence coverage from one-shot retrieval. If Top-k does not contain direct answers, decomposition often helps.

Q: How do you implement the router?
A: I start with simple rules and telemetry-driven thresholds, then move to a lightweight classifier if needed. The router outputs a small set of strategies, not open-ended plans. I evaluate router accuracy on labeled queries and measure impact on end-to-end correctness and cost.

Q: How do you prevent adaptive pipelines from looping?
A: I set a hard cap on iterations and require an improvement signal to continue. I also include timeouts and a fallback behavior: ask a clarifying question or abstain. Predictability matters more than squeezing out a small accuracy gain.

Q: How do you choose the number of sub-queries?
A: I keep it small and structured, usually 2–5 sub-queries tied to the user's intent. Each sub-query should have a clear retrieval target. If decomposition produces many sub-queries, it usually means the user question is underspecified.

Q: How do you evaluate Adaptive RAG?
A: I evaluate by query category: one-shot accuracy, decomposition accuracy, latency, and cost per query. I also track router confusion, such as complex queries misrouted to one-shot. The key is not only accuracy but also stable runtime behavior.

Q: What is a good confidence signal for stopping?
A: Coverage is the most practical: do we have at least one chunk that directly answers each sub-question. Reranker score separation is another: is the top evidence clearly better than the rest. I also use contradiction detection to avoid answering when evidence conflicts.

Q: How do you handle user questions that require a recommendation?

A: I retrieve evidence for criteria and constraints first, then separate "facts" from "judgment". I make the recommendation conditional and cite the supporting facts. If the user's preferences are missing, I ask a clarifying question rather than guessing.

Q: How does adaptive routing interact with Corrective RAG?
A: Routing can trigger corrective verification based on intent, like "latest" or regulated topics. The strategy can be "oneshot + verify" rather than full decomposition. This keeps safety checks targeted where they matter.

Q: What is the simplest adaptive improvement you would add to naive RAG?
A: A query classifier that detects "lookup" vs "procedural" vs "compare". Lookup uses Top-k retrieval; procedural uses step-focused retrieval and summarization constraints; compare uses decomposition. This single change often improves both relevance and response structure.

7.4 Interview Cheatsheet

Interview Cheatsheet

Key terms (1-line)

- **Router:** component that selects the retrieval strategy per query.
- **Decomposition:** split a complex question into answerable sub-questions.
- **Stopping criteria:** rules that end iteration to control cost and latency.
- **Coverage:** evidence exists for each required aspect of the question.
- **Misroute:** router selects a strategy that cannot answer correctly.
- **Fallback:** clarifying question or abstention when evidence is weak.

Rapid-fire rehearsal

- **Q:** Why adaptive? **A:** mixed workloads; don't run heavy pipelines on easy queries.
- **Q:** Decompose when? **A:** compare, multi-constraint, low coverage.
- **Q:** Stop rule? **A:** cap iterations + require improvement signal.
- **Q:** How evaluate router? **A:** labeled queries + cost/latency impact.

"Tell me about a time..." story skeleton

- **Situation:** A single RAG pipeline performed well on lookups but poorly on comparisons.
- **Task:** Improve accuracy while keeping cost predictable.
- **Action:** Labeled queries by strategy, implemented a router, added decomposition for complex queries, and introduced stopping criteria and fallbacks.
- **Result:** Better correctness on complex questions with controlled latency, plus clearer error handling when evidence was missing.

Chapter 8

Agentic RAG

Interview Anchor

- **Sound bite:** "Agentic RAG uses an agent to plan, call tools, retrieve from multiple sources, and iteratively refine evidence before answering."
- **Sound bite:** "I treat tools as APIs with permissions, budgets, and logs; the agent is powerful but must be constrained."
- **Sound bite:** "The key outputs are: a plan, traceable tool calls, and a grounded final answer."

Whiteboard framework (5 steps)

1. Plan: decide steps and tools (retrieval, DB, web/API, calculator, etc.).
2. Act: execute tool calls and collect evidence with provenance.
3. Observe: evaluate evidence quality; detect gaps and conflicts.
4. Iterate: refine queries or branch to other tools within a budget.
5. Answer: synthesize with citations; include an uncertainty or abstain path.

Example interview questions

- Conceptual: What makes an agentic RAG system different from adaptive routing?
- Scenario: Build a support agent that checks internal docs, a ticketing API, and recent release notes to answer customer questions. How do you design it?

In interviews, Agentic RAG is where reliability engineering and security become central. The interviewer expects you to talk about tool design, permissioning, and observability. A strong answer explains how you keep the agent deterministic enough to trust, even though it can take many actions.

Agentic RAG uses AI agents with planning and tool use to orchestrate retrieval from multiple sources. Instead of a fixed pipeline, the agent can decide to query a vector index, run a SQL query, call an API, or perform verification. This is useful for workflows that require combining evidence across systems.

In practice, the agent needs constraints to be safe and predictable. I enforce a tool budget, whitelisted tools, and input/output schemas per tool. I also store full traces: plan, tool calls, tool

results, and the final answer with citations.

Agentic RAG succeeds when evidence collection is systematic. I separate "work" messages (tool results, intermediate notes) from the final user-facing answer. The user sees a grounded answer; the system retains traces for debugging and evaluation.

8.1 Core Pipeline

8.1.1 Tool interface design

Each tool should be a stable function with typed inputs and structured outputs. For retrieval tools, outputs include document IDs, spans, and scores. For APIs, outputs include request IDs and timestamps so results are auditable.

8.1.2 Planning and execution

Planning can be explicit (the model writes a plan) or implicit (policy-driven). I prefer explicit plans with a small number of steps because they are inspectable. Execution should validate tool inputs, enforce budgets, and handle tool failures gracefully.

8.1.3 Memory and state

Agentic systems often maintain short-term state (current evidence set, hypotheses) and long-term memory (user preferences, prior resolutions). I keep memory minimal and scoped, and I avoid storing sensitive user data unless explicitly required. Memory entries should be traceable to their sources.

8.2 Implementation Notes

8.2.1 Constrained agent loop (pseudocode)

```
budget = {"tool_calls": 6, "tokens": 8000}
state = {"evidence": [], "notes": []}

plan = agent.plan(query, available_tools, budget)
for step in plan.steps:
    if budget_exhausted(budget): break
    tool = registry.get(step.tool_name)
    args = validate_schema(step.args, tool.input_schema)
    result = tool.call(args)
    state["evidence"] += normalize_evidence(result)
    state["notes"].append({"step": step, "result_meta": meta(result)})

signals = assess_evidence(query, state["evidence"])
if signals["low_coverage"] or signals["conflict"]:
    return ask_clarifying_or_abstain(query, state["evidence"])

return llm.generate(build_prompt(query, state["evidence"], require_citations=True))
```

8.2.2 Security and governance checklist

I restrict tools by role and topic, log every tool call, and redact secrets in traces. I also require deterministic timeouts and retry policies so the agent cannot hang. In production, I run agent actions in a sandboxed environment with least-privilege credentials.

8.3 Interview Questions and Answers

Q: What is Agentic RAG in one sentence?
A: Agentic RAG uses an agent to plan and execute multiple tool calls—including retrieval, databases, and APIs—to collect evidence, then generates a grounded answer with citations. It is designed for complex workflows that require iterative information gathering. The core is traceable action, not just generation.

Q: How is agentic RAG different from Adaptive RAG?
A: Adaptive RAG selects among predefined retrieval strategies, usually within one system boundary. Agentic RAG can take actions across multiple tools and systems, and it may iterate based on observations. The trade-off is higher complexity, so you need tighter controls and logs.

Q: What are the main risks of agentic systems?
A: The main risks are unsafe tool use, data leakage, and unpredictable cost or latency. Agents can also "over-act" by calling unnecessary tools. I mitigate with tool allowlists, budgets, typed schemas, and strong observability.

Q: How do you design a tool interface for retrieval?
A: I make the output structured: chunk text, `doc_id`, `span`, score, and source metadata. I also include a stable URL or pointer to the original artifact for auditing. This structure makes citations and validation straightforward.

Q: What stopping criteria do you use for an agent loop?
A: I enforce hard budgets on tool calls, time, and tokens. I also stop when coverage is sufficient or when repeated tool calls do not improve evidence quality. A clear fallback is required: ask a clarifying question or abstain.

Q: How do you prevent prompt injection through retrieved content?
A: I treat retrieved content as untrusted input and strip or fence instructions in it. I also separate system/tool instructions from retrieved text and explicitly tell the model to ignore instructions inside documents. For higher risk, I add a content classifier and allow only whitelisted fields into the prompt.

Q: How do you test agentic RAG?
A: I test tools individually with unit tests, then test the agent with scripted scenarios and golden traces. I also run regression tests on tool call sequences and final answer faithfulness. Observability is part of testing: if you cannot replay a trace, you cannot debug it.

Q: What does good observability look like for an agent?
A: I log: the plan, each tool call with inputs/outputs, evidence IDs, and the final answer with citations. I also record latency per step and reasons for stopping. These traces allow performance tuning and incident debugging.

Q: When would you choose agentic RAG over a fixed pipeline?

A: When the workflow requires multiple systems, like combining internal docs with a ticketing API and recent release notes. It is also useful when the answer depends on conditional branching, like "if account tier is X, do Y." If a fixed pipeline works, I prefer it because it is simpler to maintain.

Q: How do you handle partial tool failures?
A: I classify failures into retryable vs non-retryable and enforce bounded retries with backoff. The agent should degrade gracefully by using remaining evidence and stating limitations. Silent failures are unacceptable; they must be surfaced in logs and, when relevant, to the user.

8.4 Interview Cheatsheet

Interview Cheatsheet

Key terms (1-line)

- **Agent loop:** plan → act → observe → iterate → answer.
- **Tool schema:** typed input/output contract for safe tool calls.
- **Budget:** hard limits on tool calls, time, and tokens.
- **Trace:** logged sequence of plans, tool calls, results, and citations.
- **Least privilege:** tools run with minimal permissions needed.
- **Prompt injection:** malicious instructions embedded in retrieved content.

Rapid-fire rehearsal

- **Q:** Why agentic? **A:** multi-system workflows need iterative evidence collection.
- **Q:** Biggest risk? **A:** unsafe tool use; control with schemas + budgets + logs.
- **Q:** Key artifact? **A:** a replayable trace of actions and evidence.
- **Q:** Injection defense? **A:** treat docs as untrusted; fence instructions; whitelist fields.

"Tell me about a time..." story skeleton

- **Situation:** Customer issues required combining internal docs, tickets, and release notes.
- **Task:** Build an assistant that can gather evidence across tools and remain auditable.
- **Action:** Designed typed tool APIs, enforced budgets and allowlists, logged full traces, and implemented prompt-injection defenses and regression tests.
- **Result:** Faster support resolutions with verifiable citations and predictable runtime behavior under production constraints.

Appendix A

Interview Drill Plan (5–7 Days)

This appendix is a compact plan to rehearse the eight RAG architectures as interview-ready answers. The goal is to practice both the *systems thinking* and the *spoken delivery*. Each day includes reading, active recall, and a short "whiteboard" drill.

A.1 7-day plan

A.1.1 Day 1: Baseline retrieval fundamentals

Read Chapter 1 (Naive RAG). Rehearse the Interview Anchor sound bites until you can say them without looking. Whiteboard the 4-step pipeline and explain how you measure recall@k and faithfulness.

A.1.2 Day 2: Multimodal grounding

Read Chapter 2 (Multimodal RAG). Practice explaining OCR/ASR as ingestion and provenance as the core reliability requirement. Whiteboard late fusion vs joint embedding and give one example with timestamp or bounding-box citations.

A.1.3 Day 3: Hard-query retrieval

Read Chapter 3 (HyDE). Practice a 30-second explanation of embedding mismatch and why HyDE can improve recall. Whiteboard the "draft $\rightarrow$ embed $\rightarrow$ retrieve $\rightarrow$ verify" loop and explain topic-drift mitigation.

A.1.4 Day 4: Freshness and verification

Read Chapter 4 (Corrective RAG). Drill a conflict scenario: internal doc vs official page. Whiteboard the verification loop and describe a conflict policy and citation validity checks.

A.1.5 Day 5: Structured reasoning

Read Chapter 5 (Graph RAG). Practice explaining why the graph is an index over relationships, not the source of truth. Whiteboard k-hop traversal and provenance spans, then rehearse one multi-hop question end-to-end.

A.1.6 Day 6: Retrieval fusion

Read Chapter 6 (Hybrid RAG). Rehearse dense vs BM25 failure modes and the RRF fusion idea. Whiteboard the fusion pipeline and explain how you prove incremental lift with offline metrics.

A.1.7 Day 7: Advanced orchestration

Read Chapters 7 and 8 (Adaptive and Agentic RAG). Drill routing, stopping criteria, and tool governance. Whiteboard a constrained agent loop with budgets, tool schemas, and trace logging.

A.2 Daily 20-minute rehearsal template

1. **2 min:** Recite the 2–3 sound bites from the chapter without notes.
2. **5 min:** Whiteboard the framework (3–5 steps) and explain each step in one sentence.
3. **8 min:** Answer 3 rapid-fire questions out loud; keep answers under 20 seconds each.
4. **5 min:** Practice the STAR skeleton and map it to a real project you have done.

Terminology

This glossary defines key RAG terms in short, spoken-friendly language. Use it as a quick refresher before interviews. Each definition is intentionally brief so you can rehearse it out loud.

Term	**Definition (spoken-friendly)**
Abstention	A deliberate "I don't know" when evidence is missing or conflicting.
ANN (Approx. Nearest Neighbor)	Data structure for fast vector Top-k search without scanning all embeddings.
Answer faithfulness	The answer's claims are supported by cited evidence; no extra invented facts.
BM25	Keyword retrieval based on term frequency and inverse document frequency.
Candidate pool	Larger set retrieved before reranking and final Top-k selection.
Chunk	A retrieval unit (text span) stored with metadata and an embedding.
Chunk overlap	Repeating tokens across chunks to avoid cutting facts at boundaries.
Citation validity	A citation points to an existing snippet that actually supports the claim.
Conflict policy	Rules for what to do when sources disagree (prefer, surface, or abstain).
Cross-encoder	Model that scores relevance using both query and chunk together.
Dense retrieval	Retrieval using vector similarity between embeddings.
Document store	The system that holds raw docs, chunks, metadata, and source pointers.
Embedding	Vector representation of text or media used for similarity search.
Entity linking	Map a mention ("S3") to a canonical entity ID in a graph or catalog.
Evidence coverage	Whether retrieved context contains support for each required aspect of the question.
Extraction (OCR/ASR)	Turning images/audio into text surrogates with location pointers.
Fusion	Combining ranked lists from multiple retrievers into one candidate ranking.
Graph traversal	Following edges in a knowledge graph to collect connected facts.
Grounding	Constraining outputs to be based on retrieved evidence with citations.
Hallucination	A confident statement not supported by provided evidence.
HyDE	Generate a hypothetical document, embed it, then retrieve real sources.
Index	Data structure that maps embeddings to chunk IDs and supports Top-k search.
Intent routing	Decide which retrieval strategy or modality to use based on the query.
Joint embedding	Shared vector space that aligns text and images for cross-modal retrieval.
k-hop	Number of graph edge steps followed from a seed entity during traversal.
Latency budget	Maximum allowed runtime; used to cap retrieval depth and tool calls.
Late fusion	Merge evidence after retrieving separately per modality or retriever.
LLM-as-judge	Using an LLM to score answer quality (must be validated with spot checks).

Term	Definition (spoken-friendly)
Metadata filter	Restrict retrieval by fields like product, date, tenant, or document type.
MMR	Diversification method that reduces redundancy in retrieved chunks.
MRR	Ranking metric that rewards putting the correct evidence near the top.
Observability	Logs and traces of retrieval, scores, prompts, and outputs for debugging.
Provenance	Pointer back to the original artifact and location (doc/page/box/time).
Query decomposition	Split a complex question into sub-questions to retrieve evidence for each.
Query expansion	Add terms/paraphrases to improve retrieval coverage.
Recall@k	Whether correct evidence appears somewhere in the Top-k retrieval results.
Reranker	Model that reorders candidates to improve precision of final evidence set.
RRF	Reciprocal Rank Fusion: rank-based method to combine multiple retrievers.
Router	Component that selects a strategy (oneshot, hybrid, decompose, verify).
Score calibration	Make scores from different retrievers comparable; often non-trivial.
Source registry	Ranked allowlist of trusted sources used for verification.
Span	Character/token offsets that locate a claim inside a document chunk.
Tool schema	Typed contract for tool inputs/outputs to keep agents safe.
Trace	Replayable record of tool calls, evidence, and model outputs.
TTL cache	Cache external results for a time window to control latency and cost.
Vector database	Managed store for embeddings and ANN search with metadata filters.